



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

SECJ2013

DATA STRUCTURE AND ALGORITHM

SECTION 02

ASSIGNMENT 1

GROUP 6

MEMBER:

CHEW CHIU XIAN (A23CS0061)

HARINI A/P SANGARAN (A23CS0081)

MOHAMED ALI MOHAMED ALI (A21EC0287)

SYARIFAH DANIA BINTI SYED ABU BAKAR (A23CS0183)

LECTURER:

DR. ZURAINI BINTI ALI SHAH

Array used: {60, 70, 95, 75, 88}

1) Bubble Sort (Member: Chew Chiu Xian)

Code:

```
#include <iostream>
```

```
using namespace std;
```

```
void bubbleSort(int arr[], int n, int &comparisons, int &swaps) { // Bubble Sort Function
```

```
    comparisons = 0;
```

```
    swaps = 0;
```

```
    for (int i = 0; i < n - 1; i++) {
```

```
        for (int j = 0; j < n - i - 1; j++) {
```

```
            comparisons++;
```

```
            if (arr[j] > arr[j + 1]) { // Compare the marks
```

```
                swap(arr[j], arr[j + 1]); // Swap the position of marks
```

```
                swaps++;
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
int main() {
```

```
    int marks[] = {60, 70, 95, 75, 88}; // Student marks list
```

```
    int n = sizeof(marks) / sizeof(marks[0]);
```

```

int comparisons, swaps;

cout << "Bubble Sort" << endl;

cout << "-----" << endl;

cout << "Original marks: ";

for (int i = 0; i < n; i++) {

    cout << marks[i] << " ";

}

cout << "\nTotal number of students: " << n << endl;

bubbleSort(marks, n, comparisons, swaps);


cout << "Sorted marks: ";

for (int i = 0; i < n; i++) {

    cout << marks[i] << " ";

}

cout << endl;


cout << "Total comparisons: " << comparisons << endl; // Total number of comparisons

cout << "Total swaps: " << swaps << endl; // Total number of swaps

return 0;

}

```

Calculation Steps:

Bubble Sort:

Array = {60, 70, 95, 75, 88}

Pass 1:

- Compare 60 and 70, no swap ($60 < 70$)
- Compare 70 and 95, no swap ($70 < 95$)
- Compare 95 and 75, swap ($95 > 75$) = {60, 70, 75, 95, 88}
- Compare 95 and 88, swap ($95 > 88$) = {60, 70, 75, 88, 95}
- Number of comparison = 4
- Number of swap = 2

Pass 2:

- Compare 60 and 70, no swap ($60 < 70$)
- Compare 70 and 75, no swap ($70 < 75$)
- Compare 75 and 88, no swap ($75 < 88$)
- Number of comparison = 3
- Number of swap = 0

Pass 3:

- Compare 60 and 70, no swap ($60 < 70$)
- Compare 70 and 75, no swap ($70 < 75$)
- Number of comparison = 2
- Number of swap = 0

Pass 4:

- Compare 60 and 70, no swap ($60 < 70$)
- Number of comparison = 1
- Number of swap = 0

Final sorted array = {60, 70, 75, 88, 95}

Total comparison = 10

Total swap = 2

Output for the code:

```
Bubble Sort
-----
Original marks: 60 70 95 75 88
Total number of students: 5
Sorted marks: 60 70 75 88 95
Total comparisons: 10
Total swaps: 2
```

2) Improved Bubble Sort (Member: Mohamed Ali Mohamed Ali)

Code:

```
#include <iostream>

using namespace std;

void bubbleSortImproved(int arr[], int n, int &comparisons, int &swaps) {

    bool swapped;

    comparisons = 0;

    swaps = 0;

    for (int i = 0; i < n - 1; i++) {

        swapped = false;

        for (int j = 0; j < n - i - 1; j++) {

            comparisons++;

            if (arr[j] > arr[j + 1]) {

                swap(arr[j], arr[j + 1]);

                swaps++;

                swapped = true;

            }

        }

    }

    // If no two elements were swapped in the inner loop, array is sorted

    if (!swapped)

        break;
```

```
    }  
}
```

```
int main() {  
  
    int marks[] = {60, 70, 95, 75, 88};  
  
    int n = sizeof(marks) / sizeof(marks[0]);  
  
    int comparisons = 0, swaps = 0;  
  
    cout << "Improved bubble sort\n";  
    cout << "-----\n";  
  
    // Output original array  
    cout << "Original marks List: ";  
    for (int i = 0; i < n; i++) {  
        cout << marks[i] << " ";  
    }  
    cout << endl;  
  
    cout << "Total number of students: " << n << endl;  
  
    // Perform bubble sort  
    bubbleSortImproved(marks, n, comparisons, swaps);  
}
```

```
// Output sorted array

cout << "Sorted Marks List: ";

for (int i = 0; i < n; i++) {

    cout << marks[i] << " ";

}

cout << endl;


// Output comparisons and swaps

cout << "Total Comparisons: " << comparisons << endl;

cout << "Total Swaps: " << swaps << endl;


system("pause");

return 0;

}
```


Calculation steps:

Date: _____

Page: _____

Improved Bubble sort:

Array = {60, 70, 95, 75, 88}

Pass 1:

compare 60 and 70 (No swaps)

compare 70 and 95 (No swaps)

compare 95 and 75 (swap)

compare 95 and 88 (swap)

comparisons = 4

swaps = 2

Pass 2:

compare 60 and 70 (No swap)

compare 70 and 75 (No swap)

compare 75 and 88 (No swap)

comparisons = 3

swaps = 0

Sorted Array = {60, 70, 75, 88, 95}

total comparisons = 7

total swaps = 2

ROX

Output:

```
Improved bubble sort
-----
Original marks List: 60 70 95 75 88
Total number of students: 5
Sorted Marks List: 60 70 75 88 95
Total Comparisons: 7
Total Swaps: 2
Press any key to continue . . . ■
```

3) Selection Sort (Member: Syarifah Dania)

Code:

```
#include <iostream>

using namespace std;

// Function to perform selection sort and track comparisons and swaps
void selectionSort(int arr[], int size, int &comparisons, int &swaps) {

    comparisons = 0; // Initialize comparison count

    swaps = 0;      // Initialize swap count

    for (int i = 0; i < size - 1; i++) {

        int minIndex = i; // Assume current index has the smallest value

        // Find the smallest value in the unsorted portion
        for (int j = i + 1; j < size; j++) {

            comparisons++; // Increment comparisons counter

            if (arr[j] < arr[minIndex]) {

                minIndex = j; // Update index of smallest value

            }

        }

        // Swap the smallest value with the current element
        if (minIndex != i) {
```

```

        swap(arr[i], arr[minIndex]);

        swaps++; // Increment swaps counter
    }

}

}

// Function to print an array

void printArray(const int arr[], int size) {
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int main() {
    // Input array of student marks
    int marks[] = {60, 70, 95, 75, 88};
    int size = sizeof(marks) / sizeof(marks[0]);

    // Variables to track the number of comparisons and swaps
    int comparisons, swaps;

    // Display the original array

```

```
cout << "Original list of student marks: ";  
  
printArray(marks, size);  
  
// Sort the array using selection sort  
  
selectionSort(marks, size, comparisons, swaps);  
  
// Display the sorted array  
  
cout << "Sorted list of student marks: ";  
  
printArray(marks, size);  
  
// Display the total number of comparisons and swaps  
  
cout << "Total comparisons: " << comparisons << endl;  
  
cout << "Total swaps: " << swaps << endl;  
  
return 0;  
  
}
```

Calculation steps:

selection sort calculation

Array : { 75, 95, 60, 88, 70 }

Start - unsorted : { 60, 70, 95, 75, 88 }

Pass 1: Find the largest element and put at the last index (95)

60, 70, 95, 75, 88
60, 70, 88, 75, 95

swap 88 & 95

Pass 2: Find the 2nd largest element and put at the second last index (88)

60, 70, 88, 75, 95
60, 70, 75, 88, 95

swap 75 & 88

Pass 3: Find the next largest element and put at the current last index. (75)

60, 70, 75, 88, 95
60, 70, 75, 88, 95

no swap

Pass 4: Find the next largest element and put at the current last index. (70)

60, 70, 75, 88, 95
60, 70, 75, 88, 95

no swap

Pass 5: Find the next largest element and put at the last index. (60)

60, 70, 75, 88, 95
60, 70, 75, 88, 95

no swap.

End → sorted

60, 70, 75, 88, 95

Output:

```
Original list of student marks: 60 70 95 75 88
Sorted list of student marks: 60 70 75 88 95
Total comparisons: 10
Total swaps: 2

-----
Process exited after 1.284 seconds with return value 0
Press any key to continue . . .
```

4) Insertion Sort (Member: Harini)

Code:

```
#include <iostream>
using namespace std;

void insertionSort(int array[], int n, int &totalComparison, int &swap) {
    int data, pass, insertIndex;
    int numOfTrueComparison = 0;

    for(pass = 1; pass < n; pass++) {
        //to compare the data from index pass until the end of array
        data = array[pass];

        //to determine the index where the largest data will move to
        insertIndex = pass;

        while((insertIndex > 0) && (array[insertIndex-1] > data)) { //comparison
            array[insertIndex] = array[insertIndex-1];    //swapping
            swap++;
            insertIndex--;
            numOfTrueComparison++;
        }
        totalComparison = totalComparison + numOfTrueComparison + 1;
        numOfTrueComparison = 0;
        array[insertIndex] = data;    //insert item at the right place
    } //end for
}

int main() {
    int marks[] = {60, 70, 95, 75, 88};
    int n = 5;
```



```
int totalComparison = 0, swap = 0;

cout << "Insertion Sort" << endl;
cout << "-----" << endl;
cout << "Original Unsorted Marks : ";
for (int i = 0; i < n; i++)
    cout << marks[i] << " ";
cout << endl;

insertionSort(marks, n, totalComparison, swap);

cout << "Sorted Marks : ";
for (int i = 0; i < n; i++)
    cout << marks[i] << " ";
cout << endl;

cout << "Total comparisons : " << totalComparison << endl;
cout << "Total swaps : " << swap << endl;
return 0;
}
```

Calculation steps:

iv) Insertion Sort

Array : { 60, 70, 95, 75, 88 }

Pass 1:

data = 70, insertIndex = 1

((1 > 0) && (60 > 70)) // False, Comparison = 1

data[1] = 70

Updated marks = 60, 70, 95, 75, 88

Pass 2:

data = 95, insertIndex = 2

((2 > 0) && (70 > 95)) // False, Comparison = 2

data[2] = 95

Updated marks = 60, 70, 95, 75, 88

Pass 3:

data = 75, insertIndex = 3

((3 > 0) && (95 > 75)) // True, Comparison = 3

data[3] = 95 // Swap = 3

insertIndex = 2

((2 > 0) && (70 > 75)) // False, Comparison = 4

data[2] = 88

Updated marks = 60, 70, 75, 95, 88

Pass 4:

data = 88, insertIndex = 4

((4 > 0) && (95 > 88)) // True, Comparison = 5

data[4] = 95 // Swap = 2

insertIndex = 3

((3 > 0) && (75 > 88)) // False, Comparison = 6

data[3] = 88

Updated marks = 60, 70, 75, 88, 95

Final Sorted Array : 60, 70, 75, 88, 95

Output:

Insertion Sort

Original Unsorted Marks : 60 70 95 75 88

Sorted Marks : 60 70 75 88 95

Total comparisons : 6

Total swaps : 2

Comparison table

Sorting Algorithm	Comparison	Swap
Bubble Sort	10	2
Improved Bubble Sort	7	2
Selection Sort	10	2
Insertion Sort	6	2

Discussion

Bubble sort has a total of 10 comparisons and 2 swaps whereas improved bubble sort has 7 comparisons and 2 swaps. Improved bubble sort is much more efficient than bubble sort since it is 30% faster to sort the array of marks. Selection sort and bubble sort have the same efficiency as both sorts have the same number of comparisons and swaps. Insertion sort is the most efficient sort as it is 40% faster than the other sorts.

Reflection

Chew Chiu Xian

From Assignment 1, I have learnt that there are multiple sorting algorithms that can be used to sort the array such as bubble sort, improved bubble sort, selection sort and insertion sort. Each sorting algorithm has its own strengths and weaknesses and the uses of the algorithms are depending on the size of the dataset. The comparison done can help us to understand the importance of algorithm thinking in computer science.

Syarifah Dania

Assignment 1 provided me with a comprehensive understanding of sorting algorithms, their mechanisms, and applications. It highlighted the significance of sorting in organizing data efficiently and solving computational problems. I now feel confident in analyzing problems and selecting the most appropriate algorithm to achieve optimal performance.

Mohamed Ali Mohamed Ali

This assignment was a helpful experience that helped me understand some sorting algorithms and compare their results. By having to apply the theoretical aspect in a practical assignment it was a good way to help me and my teammates see a real life example. I would like to give my sincere gratitude to our lecturer and my great teammates who were very friendly and helpful. I hope we did all that is required of us to complete this assignment and I look forward to more assignments and activities full of knowledge and experience.

Harini

By doing the questions in assignment 1, I got a better understanding of each concept. This assignment came in handy as the test is around the corner and I'm much more confident in using each sorting method now. My teammates have been responsive and also doing their assigned tasks on time, so it's been a pleasure working with them.